Databases

William Schultz

June 3, 2026

1 Joins

At a high level, database (i.e. SQL) tables can be viewed as n-ary relations (or, more plainly, as "spreadsheets"). For example, consider the following relation P (representing a set of homeowners)

Name	**Age**	**HouseId**
Alice	31	6
Bob	32	3
Jane	25	4

and another relation H (representing a set of houses)

Id	**Year**
6	1904
4	1965

We can consider the *cross product* of these two relations $P \times H$, which is simply the Cartesian product of all rows (i.e. tuples) in P and all rows in H, giving relation $P \times H$ as

Name	**Age**	**HouseId**	**Id**	**Year**
Alice	31	6	6	1904
Alice	31	6	4	1965
Bob	32	3	6	1904
Bob	32	3	4	1965
Jane	25	4	6	1904
Jane	25	4	4	1965

with a total tuple count of $|P \times H| = |P| \times |H| = 6$. On its own, the full cross product of two tables may be quite large and in standard SQL terminology is referred to as a *cross join* (`CROSS JOIN`).

Inner Joins

It is more commonly useful to apply some filter (e.g. via some predicate) to the tuples that are generated as a result of this cross product operation, which is commonly referred to as the *inner join* (`INNER JOIN`). That is, if we filter the result $P \times H$ based on the predicate $P.HouseId = H.Id$, then we say we're "joining" the two tables, P and H, on the predicate $P.HouseId = H.Id$, which gives as a result:

Name	**Age**	**HouseId**	**Id**	**Year**
Alice	31	6	6	1904
Jane	25	4	4	1965

which is basically the set of people in P associated with the house in H they own. More compactly, we typically notate a join on predicate p between two relations A and B as

$$A \bowtie_p B$$

and we can also just think of this as a composition of cross product and filtering operations i.e.

$$A \bowtie_p B = \sigma_p(A \times B)$$

where σ_p represents the filtering operation for a given predicate p on tuples.

Outer Joins

There are other variations of joins that are not symmetric in the same way as inner joins. Outer joins may allow you to, for example, return all rows from the "left" table of the join, matched on some predicate with rows of the "right" table, filling in NULL entries for the right table columns for rows where the predicate is unmatched (`LEFT OUTER JOIN`). For example, for the above table P and H, a left outer join on $P.HouseId = H.Id$ would give us:

Name	**Age**	**HouseId**	**Id**	**Year**
Alice	31	6	6	1904
Bob	32	3	`NULL`	`NULL`
Jane	25	4	4	1965

where the "Bob" row in the left table has NULL entries for the columns in the right table, since no rows matched the join predicate.

Similarly, a right outer join (`RIGHT OUTER JOIN`) would give us:

Name	**Age**	**HouseId**	**Id**	**Year**
Alice	31	6	6	1904
Jane	25	4	4	1965

where every row in the right table found a matching row based on the join predicate, so no columns needed to be filled in with `NULL`. If, however, table H contained a row with no match in P e.g. if H was instead

Id	**Year**
6	1904
4	1965
1	1965

then a right outer join would give us:

Name	**Age**	**HouseId**	**Id**	**Year**
Alice	31	6	6	1904
Jane	25	4	4	1965
`NULL`	`NULL`	`NULL`	1	1965

where `NULL` will fill in the columns in the left table for the row with no match in P.

Properties of Joins

Note that inner joins are *commutative* i.e. $A \bowtie_p B = B \bowtie_p A$. This can be easily seen from examining the decomposed form of joins in terms of cross products and filtering i.e. since $A \times B = B \times A$ (if we ignore ordering of columns in the output tuples). Note also that we can view a sequence of joins as a big cross product followed by a filtering operation at the end i.e.

$$(A \bowtie_p B) \bowtie_q C = \sigma_q(\sigma_p(A \times B) \times C) = \sigma_{p \wedge q}(A \times B \times C)$$

Furthermore, joins are *associative*. This means that we can re-order joins as we please (since they are both *associative* and *commutative*), so there may be many different join executing orderings that produce the same final result. That is,

$$\begin{gathered}(A \bowtie_p B) \bowtie_q C \\ A \bowtie_p (B \bowtie_q C) \\ A \bowtie_p (C \bowtie_q B) \\ (A \bowtie_p C) \bowtie_q B \\ (C \bowtie_p A) \bowtie_q B\end{gathered}$$

are all equivalent.

Joins in MongoDB

Note that the type of left outer join described above is similarly implemented as the `$lookup` stage in MongoDB aggregation pipelines. That is, you can take a set of input documents and execute a left outer join on another collection by specifying some field to join on, and the matching documents will be embedded as a sub-object with a specified field name in the original set of input documents. For example, if you have the following two collections

```
homeowners = [
    {"name": "Alice", "age": 31, "house_id": 6},
    {"name": "Bob", "age": 32, "house_id": 3},
    {"name": "Jane", "age": 25, "house_id": 4}
]

houses = [
    {"_id": 6, "year": 1904},
    {"_id": 4, "year": 1965},
    {"_id": 1, "year": 1965}
]
```

you can execute the following aggregation pipeline stage:

```
pipeline = [
    {
        "$lookup": {
            "from": "houses",           # collection to join with
            "localField": "house_id",   # field from homeowners collection
            "foreignField": "_id",      # field from houses collection
            "as": "house_info"          # array field to store the joined data
        }
    }
]
```

which will produce a result like the following:

```
{
  '_id': ..., 'name':
  'Alice', 'age': 31,
  'house_id': 6,
  'house_info': [{'_id': 6, 'year': 1904}]
}
{
  '_id': ..., 'name':
  'Bob', 'age': 32,
  'house_id': 3,
  'house_info': []
}
{
  '_id': ...,
  'name': 'Jane',
  'age': 25,
  'house_id': 4,
  'house_info': [{'_id': 4, 'year': 1965}]
}
```

Join Order Optimization

Though joins are commutative and associative, some join orderings may be much more efficient to execute. More generally, we can think of a particular join ordering as a binary tree, that essentially maps to the syntactic parse tree of the relational expressions above. More specifically, if we consider a basic example like considering the tradeoff of two different orderings:

$$(A \bowtie_p B) \bowtie_q C$$

and

$$A \bowtie_p (B \bowtie_q C)$$

then we may want to consider the tradeoff in cardinalities between $(A \bowtie_p B)$ and $(B \bowtie_q C)$.

For example, if we imagine that all tables A, B, C are roughly the same cardinality, if $|A \bowtie_p B| \gg |B \bowtie_q C|$, then it would seem that executing the second ordering may be much more efficient, since the "intermediate table" produced as part of this query plan is much smaller, and so its cross product with the third table will be small, leaving fewer rows to filter out for the final join result. This is the basic idea behind *cost-based optimization* in modern relational database systems.

Note that this type of query optimization has a long history, dating back at least to System R in the 1970s [SAC+79]. This paper also provides a good modern treatment of the basic architecture of query optimizers [LGM+15]. Also, note that join ordering considerations are very similar to the problem of matrix chain multiplication, a similar cost-based optimization problem related to order of matrix multiplication operations [HS84].

2 Data Modeling

In relational database design, there are two main approaches to data modeling (1) normalization theory and (2) entity-relationship (ER) modeling followed by ER-to-relational translation.

2.1 Normalization Theory

We may have, for example, a database that needs to store *customers*, *products*, and the *orders* placed by customers. One natural approach may be to store each entity in a separate table e.g. one table for customers, products, and orders, respectively. There are various tradeoffs, but which is a "good design"? One goal is to avoid redundancy i.e. avoid storing the same fact multiple times in different locations.

Functional dependencies (FDs) provide a formal notation for capturing such facts. The FD $custid \rightarrow name$ captrues the fact that a given customer id should only have one name assciated with it in the database, so whereever those fields appear together, the name associated with that customer should be the same. Given a set of FDs and a proposed database design, normalization theory provides a way to arrive at a design that minimizes redundancy.

Table 1: Dependency-Driven Relational Database Design

Normal Form	Description
1NF	All fields must be atomic (*i.e.,* scalar)
2NF	Avoid partial dependencies on PK (using FDs)
3NF	Avoid transitive dependencies on PK (using FDs)
BCNF	Avoid "all" remaining redundancy (using FDs)
4NF, 5NF (PJNF)	Avoid "nesting" redundancies (using MVDs)

Figure 1: Normalization Theory

Table 1 shows standard "normal forms" along with the kinds of FD-related problems that they avoid. 1NF is unique in that it is simply the relational models rule that each field in a table must be atomic (a scalar value). 2NF avoids situtations where a table has a composite key, like a primary key (PK) of $(custid, itemno)$ and has non-key fiels that are involved in facts (FDs) that depend on some but not all of the PK fields.

In general, the process of *normalization* is about structuring data in accordance with a variety of these normal forms in order to reduce data redundancy. *Denormalization*, then, is generally the process of adding redundant copies of data e.g. by grouping data, perhaps for increasing (e.g. read) performance.

2.2 Entity Relationship Modeling

Entity-Relationship model from the 1970s offers an alternative path to arriving at a "good" relational database design [Che76]. It involves capturing the conceptual data model for an application and then translating this into a logical (i.e. relational) one. The ER model for an app expresses the needed information in terms of *entities* and *relationships*, and *attributes*. Figure 2 shows a simple ER model for a basic commerce app, with *customers*, *orders*, and *products* entity sets.

The basic idea then is that you can take an ER model and translate it into a relational schema using basic rules as shown in Figure 3.

3 Transactions and Isolation

Standard model of Adya defines transaction isolation in terms of adbsence of various anomalies/cycles with a serialization graph, which consists of committed transactions as nodes and edges between them representing certain types of data dependencies between transactions.

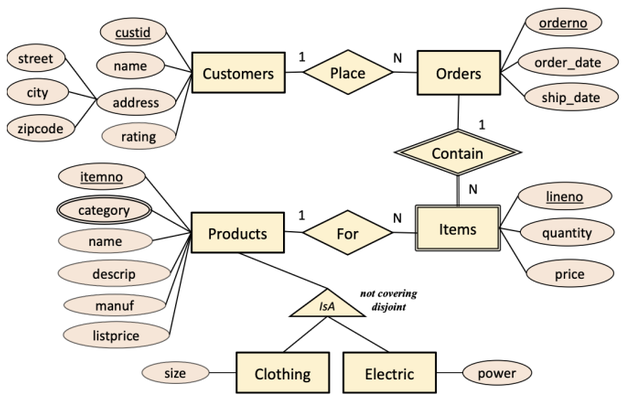

Figure 2: Extended ER Model for a Commerce Example.

Figure 2: Entity Relationship Modeling

Table 2: ER-Driven Relational Database Design

ER Concept	Relational Artifact
Entity	Table with entity's attributes and PK
Relationship (M:N)	Table with relationship's attributes and FKs
Relationship (1:N)	Merge relationship table into N-side's entity table
Composite attribute	Use a flattened column naming convention
Multivalued attribute	Add separate side table with entity's PK as FK
Inheritance hierarchy	Delta tables or mashup table

Figure 3: ER-to-relational translation

- $T_i \overset{ww}{\rightarrow} T_j$: Transaction T_j writes the version of a key k immediately following the version written by transaction T_i.
- $T_i \overset{rw}{\rightarrow} T_j$: Transaction T_j writes the next version of a key k that was read by T_i.
- $T_i \overset{wr}{\rightarrow} T_j$: Transaction T_j reads the version of key k that was written by T_i.

Snapshot Isolation

Originally defined operationally in [BBG+95], but also given a more declarative treatment in Adya's thesis [Ady99].

Serializable Snapshot Isolation

Cahill and Fekete laid down the foundations for making snapshot isolation serializable [FLO+05, CRF09], by showing that under snapshot isolation, remaining anomalies will consist specifically of cycles that contain 2 adjacent *rw* edges, which are referred to as "dangerous structures", and Cahill proposed concrete algorithm for detecting and aborting when these dangerous structures are detected.

Predicate Reads

Next-key locking [Moh90], SILO [TZK+13].

References

[Ady99] A. Adya. Weak consistency: A generalized theory and optimistic implementations for distributed transactions. Technical report, USA, 1999.

[BBG+95] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O'Neil, and Patrick O'Neil. A critique of ansi sql isolation levels. *SIGMOD Rec.*, 24(2):1–10, May 1995.

[Che76] Peter Pin-Shan Chen. The entity-relationship model—toward a unified view of data. *ACM Trans. Database Syst.*, 1(1):9–36, March 1976.

[CRF09] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. Serializable isolation for snapshot databases. *ACM Trans. Database Syst.*, 34(4), December 2009.

[FLO+05] Alan Fekete, Dimitrios Liarokapis, Elizabeth O'Neil, Patrick O'Neil, and Dennis Shasha. Making snapshot isolation serializable. *ACM Trans. Database Syst.*, 30(2):492–528, June 2005.

[HS84] T. C. Hu and M. T. Shing. Computation of matrix chain products. part ii. *SIAM Journal on Computing*, 13(2):228–251, 1984.

[LGM+15] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proc. VLDB Endow.*, 9(3):204–215, November 2015.

[Moh90] C. Mohan. Aries/kvl: a key-value locking method for concurrency control of multiaction transactions operating on b-tree indexes. In *Proceedings of the Sixteenth International Conference on Very Large Databases*, page 392–405, San Francisco, CA, USA, 1990. Morgan Kaufmann Publishers Inc.

[OCGO96] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (lsm-tree). *Acta Inf.*, 33(4):351–385, June 1996.

[SAC+79] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, SIGMOD '79, page 23–34, New York, NY, USA, 1979. Association for Computing Machinery.

[TZK+13] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, page 18–32, New York, NY, USA, 2013. Association for Computing Machinery.

4 Storage

4.1 LSM Trees

LSM Trees [OCGO96] is a persistent data structure that is optimized for writes. Essentially, it is serving the same purpose as a B-tree or hash map i.e. storing a key-value pairs of data, but is designed to be optimized for write performance.

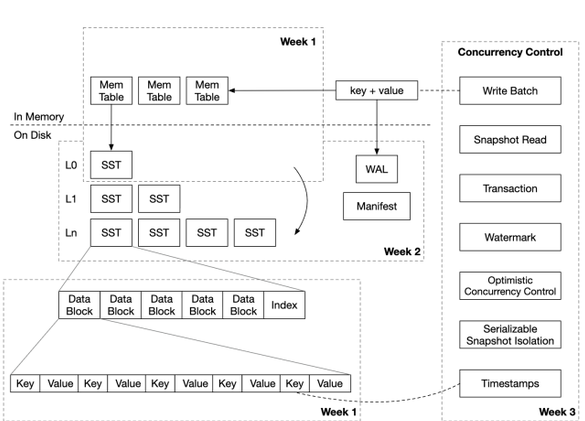

Figure 4: Log-Structured Merge Tree (LSM Tree) structure

LSM trees are kind of inspired by this basic concept with a bit of extra structure. In the simplest setup, the quickest way to insert a key-value pair into a data structure is to basically just append it to some file or log, without any extra indexing work. In the most naive approach, when you go back to search for this key, you then just do a full scan through all existing keys to see if you can find it. This is inefficient for reads but makes writes extremely simple and efficient.

In the simplest LSM tree configuration, we can imagine it being composed of two levels, C_0 and C_1, where we can imagine C_0 lives entirely in memory and C_1 is a disk-based component. When new records are inserted, they are added to level 0 component, and if

an insertion causes this component to exceed some threshold size, a contiguous segment of entries from this is removed and merged into the disk-based component, C_1.The level 0 component is maintained in memory and can be stored using a skip list or even just a B-tree.

Note that each level in an LSM tree might have multiple sorted *runs*. On disk, these are typically referred to as SSTs (Sorted String Tables). All SST files are immutable, and may exist at one of many levels of the LSM tree on disk.

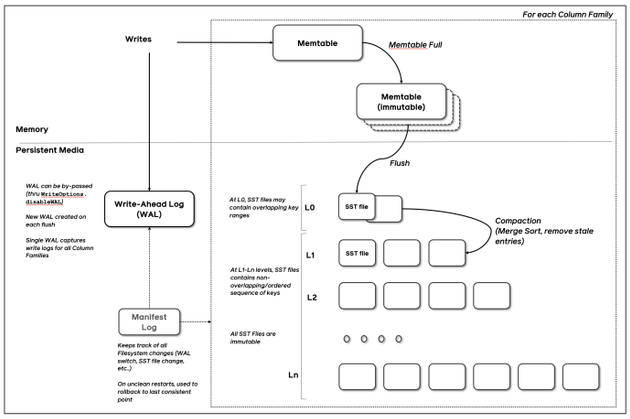

Figure 5: Log-Structured Merge Tree (LSM Tree) structure

Reads

When you go to look up a key, you're basically searching for a key by looking through either runs in the memtable or SST files at each level, working your way down the levels until you find the desired key. This can be optimized at each layer with other auxiliary data structures e.g Bloom filters, etc.